

PROJECT REPORT

Student Internship & Skill Tracker

Full Stack Development Internship Project

Submitted as part of the UpSkill Internship Program

Student Name: Prince Kumar

Project Type: Full Stack Web Application

Technology: HTML, CSS, JavaScript, Node.js, Express.js, MongoDB

2026

1. Abstract

The Student Internship & Skill Tracker is a full-stack web application designed to help students maintain and organize important career-development information in one place. The application allows users to register and securely log in, manage internship records, maintain project information, track technical skills, store certificate details, and update their personal profile. A dashboard provides a quick overview of the user's records. The project demonstrates the integration of a responsive frontend with a Node.js and Express.js backend and a MongoDB database through REST APIs.

2. Introduction

Students often maintain internship details, project information, skills, and certificates across different documents or platforms. This makes it difficult to keep career information organized and updated. The proposed system provides a centralized web application where users can store and manage these records. The project was developed as a practical full-stack development application to demonstrate frontend design, backend API development, database operations, authentication, and CRUD functionality.

3. Problem Statement

There is a need for a simple platform that allows students to manage their professional-development records in a structured way. Manual record keeping can result in scattered information, duplicate entries, and difficulty presenting an updated profile. The system addresses this problem by providing a single dashboard and database-backed application for managing internships, projects, skills, certificates, and profile information.

4. Objectives

- Develop a functional full-stack web application.
- Implement user registration and login.
- Secure user-specific resources using JWT authentication.
- Implement CRUD operations for internships, projects, skills, and certificates.
- Store application data in MongoDB.
- Provide a responsive and easy-to-use interface.
- Demonstrate REST API communication between frontend and backend.
- Provide a dashboard showing useful summary statistics.

5. Scope of the Project

The current system focuses on individual student career tracking. It can be used to maintain internship history, project details, technical skills, certificates, and basic profile information. The architecture can later be extended with features such as public portfolios, resume generation, skill assessments, notifications, admin management, and analytics.

6. Technologies Used

HTML5: Used to create the structure of web pages.

CSS3: Used for styling, layout, responsiveness, and visual presentation.

JavaScript: Used for frontend interaction and API communication.

Bootstrap: Used for responsive UI components and layout.

Node.js: Used as the server-side JavaScript runtime.

Express.js: Used to create REST APIs and backend routes.

MongoDB: Used as the NoSQL database for storing application records.

Mongoose: Used for MongoDB data modeling and database operations.

JWT: Used for authentication and protected API access.

bcrypt: Used to hash user passwords before storing them.

7. System Modules

Authentication Module: Provides registration and login functionality. Passwords are hashed and successful login generates a JWT token.

Dashboard Module: Displays the logged-in user's name and counts of internships, projects, skills, and certificates.

Internship Module: Allows users to add, view, and delete internship records, including company, position, dates, status, and description.

Project Module: Stores project title, description, technologies, GitHub link, and live-demo link.

Skills Module: Allows users to record technical skills and proficiency levels such as Beginner, Intermediate, and Advanced.

Certificate Module: Stores certificate title, issuing organization, date, and certificate link.

Profile Module: Allows users to maintain their name, bio, phone number, GitHub, and LinkedIn information.

8. System Architecture

The application follows a client-server architecture. The frontend sends HTTP requests to the Express.js backend. The backend validates authentication, processes requests, and communicates with MongoDB through Mongoose. JWT tokens are sent with protected requests so that users can access only their own records.

9. Database Design

The database contains separate collections for users, internships, projects, skills, and certificates. Each career-related record stores a reference to the user who created it. This allows the backend to filter records by authenticated user and prevents users from directly accessing another user's records through the normal API routes.

10. API Design

The backend exposes REST-style endpoints. Authentication uses POST requests for registration and login. Profile information can be retrieved with GET and updated with PUT. Internship, project, skill, and

certificate modules provide GET and POST endpoints, along with PUT and DELETE endpoints for individual records. Protected routes require a valid Bearer JWT token in the Authorization header.

11. Security

User passwords are not stored as plain text; bcrypt is used for password hashing. JWT authentication is used to protect application APIs. The backend obtains the authenticated user ID from the verified token and associates created records with that user. Update and delete operations also check ownership before modifying a record.

12. User Workflow

1. User opens the application.
2. New user creates an account using the registration page.
3. User logs in with email and password.
4. The server verifies credentials and returns a JWT token.
5. User enters the dashboard.
6. User adds internships, projects, skills, and certificates.
7. Records are sent through REST APIs and stored in MongoDB.
8. Dashboard statistics update according to stored records.
9. User can manage their profile and remove records when required.

13. Testing

The application should be tested module by module. Registration and login should be tested with valid and invalid credentials. CRUD operations should be tested by creating, viewing, updating, and deleting records. Protected routes should be checked without a token and with an invalid token. Database persistence should be verified by restarting the server and checking that stored records remain available.

14. Advantages

- Centralized student career information.
- Simple and responsive interface.
- Secure authentication mechanism.
- Database-backed persistent storage.
- REST API based architecture.
- Modular backend structure.
- Demonstrates practical full-stack development concepts.

15. Future Enhancements

The project can be extended with a React frontend, admin dashboard, public student portfolio URLs, resume generation, file uploads for certificates, skill assessment quizzes, charts and analytics, email notifications, cloud deployment, and an AI-based career recommendation module. These features are not required for the basic internship version but provide clear directions for future development.

16. Conclusion

The Student Internship & Skill Tracker successfully demonstrates the fundamental concepts of full-stack web development. The project combines a frontend interface, backend REST APIs, MongoDB data storage, authentication, and CRUD operations into one application. It provides a practical solution for organizing student career information while giving hands-on experience with modern web development technologies.

17. Project Folder Structure

student-internship-skill-tracker/

- ■ ■ ■ frontend/
 - ■ ■ ■ index.html
 - ■ ■ ■ login.html
 - ■ ■ ■ register.html
 - ■ ■ ■ dashboard.html
 - ■ ■ ■ internships.html
 - ■ ■ ■ projects.html
 - ■ ■ ■ skills.html
 - ■ ■ ■ certificates.html
 - ■ ■ ■ profile.html
 - ■ ■ ■ css/style.css
 - ■ ■ ■ js/
- ■ ■ ■ backend/
 - ■ ■ ■ server.js
 - ■ ■ ■ package.json
 - ■ ■ ■ models/
 - ■ ■ ■ routes/
 - ■ ■ ■ middleware/
- ■ ■ ■ .gitignore
- ■ ■ ■ README.md

18. Key Learning Outcomes

Through this project, the developer gains practical experience in designing web pages, handling forms, implementing client-server communication, building REST APIs, performing database operations, implementing authentication, organizing backend code into routes and models, and testing a complete full-stack workflow.